

Reinforcement Learning for Sim-to-Sim Robot Transfer

Leonardo Passafiume (358616)

Lucio Baiocchi
Thomas Kelly

Maurizio Lanzoni Raiteri

Abstract

This report studies reinforcement learning for continuous control and sim-to-real transfer in the simplified sim-to-sim setting proposed by the project. First, we implemented REINFORCE, REINFORCE with a constant baseline, and an actor-critic policy-gradient variant from scratch on Hopper-v4. Then, we moved to the PandaPush-v3 manipulation task and trained Stable-Baselines3 PPO and SAC agents under source and target dynamics, where the target object is heavier than the source object. The final implementation includes an evaluation for fixed and randomized object mass and friction, a reward-shaping wrapper, Uniform Domain Randomization (UDR) over mass and friction, and an Adaptive Domain Randomization (ADR) wrapper. The main results obtained for the PandaPush-v3 task show that a well tuned SAC policy can transfer quite well under low sim-to-real changes. However, when the sim-to-real gap is marked (hard evaluation setting) vanilla SAC fails and Domain Randomization (DR) are needed to close the gap.

1. Introduction

Reinforcement learning (RL) formulates control as sequential decision making, where an agent learns a policy by interacting with an environment and maximizing expected return [9]. In robotics this setting is attractive because it can optimize behaviors that are difficult to hand-design, but it is also difficult because interactions are expensive, exploration can be unsafe, and the learned policy can exploit simulator-specific details [4, 5]. A common practical solution is to train in simulation and transfer the policy to the real robot. The central difficulty is the reality gap: the simulator dynamics are never exactly equal to the deployment dynamics [3].

This project follows the course sim-to-real abstraction in a controlled sim-to-sim benchmark. In Part 1, we implement basic policy-gradient algorithms on Hopper-v4 to study the variance and stability issues behind direct policy optimization. In Part 2, we use PandaPush-v3, where the source domain has a light object and the target domain

has a heavier object. The target domain plays the role of the real world: it is available for evaluation, but training directly on it is treated as an oracle upper bound rather than as the deployable solution.

The second part uses modern actor-critic algorithms from Stable-Baselines3. PPO constrains policy updates through a clipped surrogate objective [8], while SAC is an off-policy maximum-entropy method that learns stochastic policies and reuses data through a replay buffer [2]. To reduce the transfer gap, we implement domain randomization, which trains over a distribution of simulator parameters instead of a single nominal model [6, 7, 10]. The implemented UDR wrapper samples parameters from fixed ranges, while the ADR wrapper adapts the mass range based on recent success, following the curriculum idea behind automatic domain randomization [1].

2. Methodology

2.1. Part 1: Policy Gradients on Hopper

The first part uses Hopper-v4, a continuous-control MuJoCo task. The action is continuous and the policy is stochastic. In `part1/agent.py`, the policy network is a two-hidden-layer MLP with 64 units per layer and `tanh` activations. The actor outputs the mean of a diagonal Gaussian distribution; the standard deviation is a learned parameter passed through `softplus`. All Part 1 experiments use Adam with learning rate 3×10^{-4} and discount factor $\gamma = 0.99$.

REINFORCE. For an episode trajectory, the implementation computes Monte-Carlo returns

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k, \quad (1)$$

and minimizes the negative policy-gradient objective

$$\mathcal{L}_{PG} = - \sum_t (G_t - b) \log \pi_\theta(a_t | s_t), \quad (2)$$

where b is either zero or a constant baseline.

Actor-Critic Variant. The actor-critic implementation uses the same actor network plus a critic network that predicts $V_\phi(s)$. At each update, the critic is trained by regression to the one-step bootstrapped target

$$y_t = r_t + \gamma V_\phi(s_{t+1})(1 - d_t), \quad (3)$$

where d_t indicates episode termination and the next-state value is detached from the gradient. The actor uses the corresponding advantage estimate

$$A_t = y_t - V_\phi(s_t), \quad (4)$$

which is normalized over the collected episode before computing the policy-gradient loss.

2.2. Part 2: PandaPush Pipeline

The second part uses the `panda-gym` environment. The source object mass is 1.0 kg, while evaluation is performed on two target settings described in the evaluation protocol. The goal position is fixed at the center of the table, while the object initial position is randomized in the xy plane. The training scripts use the dense task reward and an episode horizon of 500 steps.

Training Setting The training was performed trying different hyperparameters, and then choosing the ones that best performed. Table 1 reports the main hyperparameters used for the final SAC and PPO runs.

Hyperparameter	SAC	PPO
Training steps	500k	500k
Policy	MultiInputPolicy	MultiInputPolicy
Learning rate	$10^{-3}, 3e^{-4}, 10^{-4}$	$10^{-3}, 3e^{-4}, 10^{-4}$
Discount γ	0.95, 0.98, 0.99	0.95, 0.98, 0.99
Batch size	512, 1024, 2048	512, 1024, 2048
Gradient steps / epochs	-1	5
Rollout steps	-	2048
Learning starts	100k	-
Replay buffer size	10^6	-
Target update τ	0.05	-
Entropy coefficient	auto	0.01
GAE λ	-	0.95
Clip range	-	0.2
Policy network	default SB3	[512, 256, 128]

Table 1. Main hyperparameters used for the Part 2 Stable-Baselines3 training runs. Dashes indicate algorithm-specific parameters that are not used.

Reward Shaping. The wrapper `part2/wrappers.py` changes only the scalar training reward, not the dynamics. The shaped reward is

$$r_{\text{train}} = r_{\text{env}} - p + B\mathbf{1}[d(o, g) < \epsilon], \quad (5)$$

where:

- p is a per-step penalty,

- B is a close-goal bonus,
- $d(o, g)$ is the object-goal distance

The values used for the training are $p = 10^{-2}$, $B = 1.0$, and $\epsilon = 0.05$.

Reward shaping was added on top of the dense environment reward to encourage faster goal reaching through a small per-step penalty and an additional bonus when the object is close to the target. Evaluation uses the environment reward directly, so success rate is not computed from the shaped reward.

Domain Randomization. The wrapper `part2/rand.wrapper.py` implements three modes. In `none`, the nominal environment dynamics are kept. In `udr`, the wrapper samples a new mass and friction values at every reset. The current UDR training ranges are:

$$m \sim \mathcal{U}(1, 5), \quad \mu_{\text{obj}}, \mu_{\text{table}} \sim \mathcal{U}(0.5, 1.2), \quad (6)$$

and object spinning friction is sampled from $\mathcal{U}(0, 0.005)$. In `adr`, the wrapper starts from the nominal mass and friction and adapts the sampling ranges using a window of 20 episode outcomes. If the mean success rate is at least 0.70, the mass upper bound expands toward 5 kg and the friction ranges widen toward their global limits. If the mean success rate is at most 0.35, the ranges contract back toward the nominal values. With the default step size, the mass range expands by 1 kg per successful update, while lateral-friction ranges expand by 0.175 per update.

Evaluation Protocol. Each model is evaluated with deterministic actions for 50 episodes, using seeds 0, 42, and 99. The main metric is the final success rate, computed from `info["is_success"]` at the end of each episode. We use two target settings: an easy fixed target with the nominal target mass, and a hard target where mass and friction are randomized at test time.

Setting	Mass (kg)	Object μ	Table μ	Object spin
Easy target	5.0	0.5	0.5	0.0
Hard target	$\mathcal{U}(5, 10)$	$\mathcal{U}(0.2, 1.5)$	$\mathcal{U}(0.2, 1.5)$	$\mathcal{U}(0, 0.01)$

Table 2. Target-domain evaluation settings used for the final success-rate comparison.

3. Experimental Results

3.1. Part 1: Hopper

Figure 1 compares REINFORCE without baseline, REINFORCE with a constant baseline of 20, and the actor-critic variant over seeds 0, 42, and 99. All methods improve from the initial policy, but the curves remain noisy, as expected for policy-gradient training on Hopper. The constant

baseline gives the strongest final training curve and reduces some early variance relative to plain REINFORCE. The actor-critic variant learns more slowly and reaches lower average returns in this implementation.

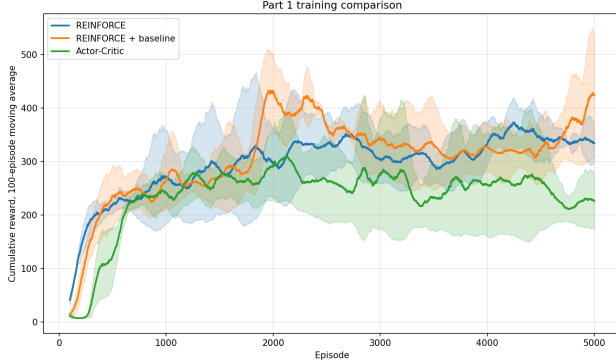


Figure 1. Part 1 training comparison on Hopper-v4. Curves show the 100-episode moving average of cumulative reward, averaged over seeds 0, 42, and 99; shaded regions show variability across seeds.

3.2. Nominal Sim-to-Sim Evaluation

Table 3 reports deterministic 50-episode evaluations generated from the saved SAC checkpoints. At the nominal target mass of 5 kg, the source-trained SAC policy already transfers well: the gap between source→source and source→target is small in success rate. The target-trained model is still the oracle upper bound.

3.3. Harder Mass and Friction Evaluation

Because nominal mass transfer was already strong, the evaluation was extended to randomize parameters at test time. Table 4 uses mass sampled uniformly in [5, 10] kg, object/table lateral friction sampled in [0.2, 1.5], and object spinning friction sampled in [0, 0.01]. Under this harder protocol, the non-randomized source policy drops from 96% to 76% success on target. The UDR policy trained with mass and friction randomization reaches 90%, suggesting that UDR is more useful when the evaluation distribution is wider than the original source-target mass shift.

Model	Mean return	Std	Success
SAC none source	-10.990	17.411	76%
SAC UDR + friction	-5.004	14.170	90%

Table 4. Hard target evaluation over 50 deterministic episodes, seed 42. The sampled masses had min 5.219 kg, mean 7.676 kg, and max 9.878 kg.

3.4. Missing Final Experiments

The following results are still required before the report can be considered final: PPO deterministic evaluation with valid normalization statistics, ADR source→source and source→target evaluation, multi-seed means and standard deviations, and the final Part 1 numeric table. **[TODO: Run the mandatory final grid and replace the TODO cells before submission.]**

4. Discussion

4.1. Result Analysis

The most important observation is that the nominal source→target transfer is easier than expected from mass alone. A source-trained SAC policy reaches 96% success on the 5 kg target object, only slightly below its 98% source success. This suggests that the current PandaPush-v3 setup is not very sensitive to the fixed 1 kg to 5 kg mass shift. The task has a fixed goal, a short object-goal distance distribution, dense rewards during training, and a powerful continuous-control algorithm. These factors can hide part of the intended reality gap.

The harder randomized evaluation is therefore more informative. When target mass is sampled in [5, 10] kg and friction is randomized, the non-randomized SAC policy falls to 76% success. This indicates that friction and heavier masses expose failures that are not visible in the nominal target evaluation. The UDR policy trained with mass and friction randomization performs worse than the nominal SAC policy on the easy fixed evaluation, but better on the hard evaluation. This is the expected robustness trade-off: UDR may sacrifice performance on the nominal dynamics in exchange for a policy that is less specialized.

SAC appears more suitable than PPO for this repository state. The available logs show high SAC training success after 500k steps, while PPO runs are less consistent and require careful treatment of vector normalization. This is coherent with the algorithms: SAC is off-policy and can reuse data through its replay buffer, whereas PPO is on-policy and discards rollouts after each update. For goal-conditioned pushing, replay-based learning is especially useful because informative transitions can be reused many times.

4.2. Reward Shaping

The reward-shaping wrapper affects training but not the evaluation success metric. The time penalty encourages short solutions, and the close-goal bonus increases the learning signal once the object reaches the success threshold. For SAC, this can increase success rate indirectly by making successful behavior easier to learn early in training. However, it also makes training return less comparable to raw evaluation return. This is why the evaluation script

Model	Evaluation	Mean return	Std return	Success
SAC source	target (easy)	-	-	-
SAC target	target (easy)	-	-	-
SAC source (Lower Bound)	target (hard)	-	-	-
SAC target (Upper Bound)	target (hard)	-	-	-
SAC UDR + friction source	target (hard)	-	-	-
SAC ADR + friction source	target (hard)	-	-	-

Table 3. Available 50-episode deterministic SAC evaluations with fixed nominal mass and fixed nominal friction, seed 42. The UDR checkpoint is the saved SAC_uds_source_500k_friction model.

reports both return and success rate on the unwrapped environment.

4.3. Methodological Critique

The main limitation is that the current result set is not yet statistically complete. Most numbers are single-seed evaluations with 50 episodes. They are useful for debugging and for identifying trends, but the final report should include mean and standard deviation across seeds where possible.

The domain-randomization implementation is also intentionally simple. UDR uses hand-selected uniform ranges, so its behavior depends strongly on those ranges. If the training range matches the evaluation range too closely, the comparison becomes too generous. The current setup avoids the most direct form of this issue by using conservative training friction ranges and a harder evaluation range, but a final study should report the exact train and test distributions clearly.

ADR is currently mass-only and uses a simple success-rate heuristic. It does not adapt friction, center of mass, action noise, observation noise, or target sampling. Therefore, any ADR result should be interpreted as a curriculum over object mass rather than as a full automatic domain-randomization method. In addition, the source-domain success rate may be a weak curriculum signal for transfer: a policy can be successful in the randomized training domain while still failing on the target distribution.

Finally, PPO evaluation needs care because some saved PPO models were trained behind VecNormalize. A valid deterministic PPO evaluation must load the corresponding normalization statistics; otherwise the policy sees observations on a scale different from training. This is marked as TODO instead of reporting unreliable numbers.

5. Conclusion

This project implemented the full path from basic policy-gradient methods on Hopper to sim-to-sim transfer experiments on PandaPush. In the current repository state, SAC is the strongest and most reliable Part 2 algorithm. At the nominal 5 kg target mass, the source-trained SAC policy already transfers well, reaching 96% success over 50 deterministic episodes, while the target-trained oracle reaches

100%. A harder mass-and-friction evaluation reveals a clearer reality gap: the non-randomized source policy drops to 76% success, while the UDR policy trained with mass and friction randomization reaches 90%. The final conclusions still require PPO, ADR, multi-seed, and Part 1 numerical evaluations to be completed.

References

- [1] Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving rubik’s cube with a robot hand. *arXiv preprint arXiv:1910.07113*, 2019. 1
- [2] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pages 1861–1870, 2018. 1
- [3] Sebastian Hofer, Kostas Bekris, Ankur Handa, Juan Camilo Gamboa, Melissa Mozifian, Florian Golemo, et al. Perspectives on sim2real transfer for robotics: A summary of the r:ss 2020 workshop. *arXiv preprint arXiv:2012.03806*, 2020. 1
- [4] Jens Kober, J. Andrew Bagnell, and Jan Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013. 1
- [5] Petar Kormushev, Sylvain Calinon, and Darwin G. Caldwell. Reinforcement learning in robotics: Applications and real-world challenges. *Robotics*, 2(3):122–148, 2013. 1
- [6] Fabio Muratore, Tim Gruner, Florian Wiese, Boris Belousov, Michael Gienger, and Jan Peters. Robot learning from randomized simulations: A review. *Frontiers in Robotics and AI*, 9, 2022. 1
- [7] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation*, pages 3803–3810, 2018. 1
- [8] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. 1
- [9] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, second edition, 2018. 1

- [10] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. *arXiv preprint arXiv:1703.06907*, 2017. [1](#)